

2. Implementions

2.1 Set Implementations

범용 구현체

Set 의 범용 구현체는 **HashSet**, **TreeSet**, **LinkedHashSet** 세 가지다.

HashSet 은 TreeSet 보다 훨씬 빠르다.

대부분의 연산이 HashSet 에서는 상수 시간이고, TreeSet 에서는 로그 시간이기 때문이다.

대신 HashSet 은 순서를 보장하지 않는다. 따라서 문서가 제시하는 선택 기준은 명확하다.

SortedSet 인터페이스의 연산이 필요하거나 값 순서 순회가 필요하다면 **TreeSet** 을, 그렇지 않으면 **HashSet** 을 쓰며, 실제로는 대부분의 경우 **HashSet** 을 사용한다.

LinkedHashSet 은 어떤 의미에서는 둘 사이의 중간이다.

연결 리스트가 관통하는 해시 테이블로 구현되어 삽입 순서 순회를 제공하며, HashSet 에 거의 근접하는 속도로 동작한다.

즉, TreeSet 이 유발하는 비용 증가를 감수하지 않으면서도 HashSet 의 순서 문제를 클라이언트가 겪지 않게 해준다.

튜닝 파라미터 : HashSet 중심

HashSet 의 순회는 **항목 수와 버킷 수의 합**에 선형이다.

- 항목 수 : number of entries
- 버킷 수 : number of buckets, 즉 용량(capacity)

이 때문에 초기 용량 설정이 양방향으로 문제가 된다.

초기 용량이 너무 높으면 공간과 시간을 모두 낭비하고,

너무 낮으면 용량을 늘려야 할 때마다 자료구조를 복사하느라 시간을 낭비한다.

초기 용량을 지정하지 않으면 기본값은 16이다.

과거에는 소수를 초기 용량으로 택하는 것이 유리했으나, 내부적으로 용량이 항상 2의 거듭제곱으로 반올림되므로 이제는 그렇지 않다.

초기 용량은 int 를 받는 생성자로 지정한다.

```
Set<String> s = new HashSet<String>(64);
```

또 다른 파라미터는 부하율(load factor)이다. 기본값은 0.75이며, 항목 수가 **용량 × 부하율** 을 넘어서면 용량이 자동으로 늘어난다.

공간 소비에 매우 민감한 경우가 아니면 **기본값을 받아들이는 것을 권장**한다고 한다.

기본 부하율을 사용하면서 **초기 용량만 지정하고 싶다면, 집합이 커질 것으로 예상하는 크기의 약 2배를 선택**하는 방안을 제시한다.

LinkedHashSet 은 HashSet 과 동일한 파라미터를 갖지만 순회 시간이 용량에 영향받지 않으며, TreeSet 은 튜닝 파라미터가 없다.

특수목적 구현체 : Special-Purpose

EnumSet 은 enum 타입을 위한 고성능 Set 구현체이다.

모든 원소가 동일한 enum 타입이어야 하며, 내부적으로 비트 벡터로 표현된다.

enum 타입의 범위에 대한 순회를 지원하고, 전통적인 비트 플래그를 타입 안전하게 대체한다.

```
for (Day d : EnumSet.range(Day.MONDAY, Day.FRIDAY))
    System.out.println(d);

EnumSet.of(Style.BOLD, Style.ITALIC);
```

CopyOnWriteArraySet 은 copy-on-write 배열로 뒷받침되는 구현체이다.

add, set, remove 등 모든 변형 연산이 배열의 **새 복사본을 만드는 방식**으로 구현되어, 잠금이 전혀 필요 없고 순회가 원소 삽입 및 삭제와 동시에 진행될 수 있다.

다만 대부분의 Set 구현체와 달리 **add, remove, contains** 가 **집합 크기에 비례하는 시간을 요구**한다. 따라서 거의 수정되지 않고 **자주 순회되는 Set**에 적합하며, 중복을 막아야 하는 이벤트 핸들러 목록 유지에 잘 맞는다.

2.2 List Implementations

리스트의 범용 구현체는 **ArrayList, LinkedList** 두 가지이다.

문서에서는 대부분 ArrayList 를 사용하게 될 것이라고 말하며, 상수 시간 위치 접근을 제공하고 빠르다고 서술한다.

각 원소마다 노드 객체를 할당할 필요가 없고, 여러 원소를 옮겨야 할 때 **System.arraycopy** 를 활용할 수 있기 때문이다.

문서에서는 ArrayList 를 **동기화 오버헤드가 없는 Vector** 로 생각하라고 표현한다.

그럼 LinkedList 는 언제 고려할까?

- 리스트 앞쪽에 원소를 자주 추가하는 경우
- 리스트를 순회하며 내부 원소를 삭제하는 경우

이 연산들은 **LinkedList** 에서 **상수 시간**, **ArrayList** 에서 **선형 시간**이다.

하지만 성능 면에서 반대급부가 존재한다.

위치 접근이 **LinkedList**에서는 **선형 시간**, **ArrayList**에서는 **상수 시간**이다.

게다가 LinkedList 는 상수 인자가 훨씬 나쁘다.

그래서 문서는 LinkedList 를 쓰려고 한다면 실제로 두 구현으로 애플리케이션 성능을 측정한 뒤 선택하라고 권고한다.

튜닝 파라미터 측면에서 ArrayList 는 초기 용량(리사이즈 전에 담을 수 있는 원소 수) 하나를 갖는다. LinkedList 는 튜닝 파라미터가 없고, 일곱 개의 선택적 연산을 가지며 Queue 인터페이스도 구현한다.

clone, addFirst, getFirst, removeFirst, addLast, getLast, removeLast

특수목적 구현체

CopyOnWriteArrayList 는 copy-on-write 배열로 뒷받침되며, 성격상 **CopyOnWriteArraySet** 과 유사하다.

순회 중에도 동기화가 불필요하고, 반복자가 **ConcurrentModificationException** 을 절대 던지지 않음이 보장된다.

즉, 변경이 드물고 순회가 잦으며 순회에 시간이 걸릴 수 있는 이벤트 핸들러 목록 유지에 적합하다.

관련 선택지

동기화가 필요한 경우, Vector 가 **Collections.synchronizedList** 로 동기화한 ArrayList 보다 약간 빠르다.

다만 Vector 에는 다수의 레거시 연산이 있으므로, 나중에 구현을 교체할 수 있으려면 반드시 List 인터페이스를 통해 조작해야 한다.

만약 리스트의 크기가 고정되어 있고 원소를 추가·삭제할 필요가 없다면 **Arrays.asList** 도 고려해 볼 수 있다.

이 메서드가 반환하는 리스트는 원본 배열에 의해 뒷받침되는 **고정 크기 리스트**이므로, **add** 나 **remove** 를 호출하면 **UnsupportedOperationException** 이 발생한다.

2.3 Map Implementations

범용 구현체

범용 구현체는 **HashMap**, **TreeMap**, **LinkedHashMap** 세 가지다.

- **SortedMap** 연산이나 키 순서 컬렉션 뷰 순회가 필요하면 **TreeMap**
- 최대 속도가 필요하고 순회 순서에 무관하면 **HashMap**
- **HashMap** 에 근접한 속도로 삽입 순서 순회를 원하면 **LinkedHashMap** 을 사용한다

문서에서는 **Set Implementations** 절의 내용이 Map 에도 그대로 적용된다고 말한다.

LinkedHashMap 은 LinkedHashMap 에 없는 두 가지 기능을 추가로 제공한다.

1. 접근 순서 정렬이다.

LinkedHashMap 을 생성할 때 삽입 순서가 아니라 **접근 순서**를 기준으로 정렬하도록 지정할 수 있다. 즉, 어떤 키에 연결된 값을 단순히 조회하기만 해도 그 키가 맵의 끝으로 이동한다.

```
// 세 번째 인자 accessOrder 를 true 로 주면 접근 순서 모드가 된다.  
new LinkedHashMap<>(16, 0.75f, true);
```

2. removeEldestEntry 메서드이다.

이를 재정의하면 새 매핑이 추가될 때 오래된 매핑을 자동으로 제거하는 정책을 부과할 수 있어, 커스텀 캐시를 쉽게 구현할 수 있다.

아래 예시는 맵을 최대 100개 항목까지 키우다가, 새 항목이 추가될 때마다 가장 오래된 항목을 제거하여 100개 상태를 유지한다.

```
private static final int MAX_ENTRIES = 100;  
  
@Override  
protected boolean removeEldestEntry(Map.Entry eldest) {  
    return size() > MAX_ENTRIES;  
}
```

이 메서드는 LinkedHashMap 을 상속한 클래스 안에서 재정의해야 동작한다.
접근 순서 모드와 함께 쓰면 곧바로 LRU 캐시가 된다.

특수목적 구현체

특수목적 구현체는 **EnumMap**, **WeakHashMap**, **IdentityHashMap** 세 가지다.

EnumMap 은 내부적으로 배열로 구현된, enum 키를 위한 고성능 Map 구현체다.

Map 인터페이스의 풍부함과 안전성을 배열에 근접한 속도와 결합한다.

문서에서는 enum 을 값에 대응시키려면 언제나 배열보다 EnumMap 을 우선 사용해야 한다고 한다.

WeakHashMap 은 키에 대한 약한 참조(weak reference)만 저장하는 구현체다.

키가 WeakHashMap 밖에서 더 이상 참조되지 않으면 해당 key-value 쌍이 GC 될 수 있다.

약한 참조의 힘을 가장 쉽게 활용하는 방법이며, 항목의 유용성이 그 키의 도달 가능성과 함께 사라지는 registry-like 자료구조 구현에 유용하다.

IdentityHashMap 은 해시 테이블 기반의 동일성(identity) 기반 Map 구현체다.

키를 비교할 때 equals 가 아니라 == 를 사용한다는 점이 핵심이다.

직렬화나 깊은 복사처럼 위상을 보존하는 객체 그래프 변환에 유용한데, 이런 변환에는 이미 방문한 객체를 추적하는 동일성 기반 노드 테이블이 필요하기 때문이다.

또한 동적 디버거 등에서 객체와 메타정보의 매핑을 유지하는 데 쓰이며, 키에 대해 equals 를 절대 호출하지 않으므로 의도적으로 왜곡된 equals 를 악용한 스푸핑 공격을 저지하는 데 유용하다.

동시성 구현체 : Concurrent

java.util.concurrent 패키지는 ConcurrentMap 인터페이스를 포함하는데, 이는 원자적 putIfAbsent(), remove(), replace() 메서드로 Map 을 확장하며 그 구현체가 **ConcurrentHashMap** 이다.

ConcurrentHashMap 은 해시 테이블로 뒷받침되는 고동시성·고성능 구현체이다.

조회를 수행할 때 절대 블로킹되지 않으며, 갱신의 동시성 수준을 클라이언트가 선택할 수 있다.

Hashtable 을 대체할 목적으로 설계되어 ConcurrentMap 구현에 더해 Hashtable 고유의 레거시 메서드도 모두 지원한다.

단, 레거시 연산이 필요 없다면 ConcurrentMap 인터페이스를 통해 조작하도록 주의해야 한다.

2.4 Queue Implementations

범용 구현체

- **LinkedList**

: 앞선 리스트 절에서 언급했듯 **LinkedList** 는 **Queue** 인터페이스를 구현하여 **add** , **poll** 등에 대해 **FIFO** 큐 연산을 제공한다.

: 별도의 큐 전용 클래스를 쓰지 않고도 **LinkedList** 하나로 리스트와 **FIFO** 큐 역할을 겸할 수 있다.

- **PriorityQueue**

: 힙 자료구조에 기반한 우선순위 큐이다.

: 큐 생성 시점에 지정된 순서에 따라 원소를 정렬하는데, 그 순서는 원소의 자연 순서이거나 명시적 **Comparator** 가 부과한 순서이다.

동작의 핵심은 조회 연산과 **head** 의 정의다.

head : 지정된 순서상 **가장 작은 원소**

PriorityQueue 와 그 반복자는 **Collection** 및 **Iterator** 인터페이스의 모든 선택적 메서드를 구현한다.

하지만 **iterator** 메서드가 제공하는 반복자는 **PriorityQueue** 의 원소를 **어떤 특정한 순서로 순회하는 것을 보장하지 않는다**.

즉, 힙의 내부 저장 순서와 논리적 우선순위는 다르다.

따라서 순서 있는 순회가 필요하다면 **Arrays.sort(pq.toArray())** 를 사용해야 한다.

동시성 구현체

java.util.concurrent 패키지는 동기화된 큐 인터페이스와 클래스 집합을 담고 있다.

BlockingQueue : Interface

- 큐를 확장하여, 원소를 꺼낼 때 큐가 비어 있지 않게 될 때까지 대기하고 원소를 저장할 때 큐에 공간이 생길 때까지 대기하는 연산을 추가한다
- 이때 **대기가 블로킹 큐의 본질**이며 아래 다섯 클래스가 이를 구현한다

유계(bounded) : 최대 몇 개나 담을지 정해져 있음

무계(unbounded) : 상한이 없음

Bounded Queue : 예를 들어 100개로 가득 차면 101번째 **put()** 은 블로킹

Unbounded Queue : 꽉 찰 일이 없으니 **put()** 이 블로킹되지 않는다

OOM(Out Of Memory) : JVM 이 필요한 메모리를 확보하지 못했을 때 던지는 예러이며, 예외가 아니라 Error 이다

LinkedBlockingQueue : 연결 노드 기반, 선택적 유계 FIFO

- 생성자에 용량을 주지 않으면 `Integer.MAX_VALUE` 이며 사실상 무제한이다
- `Executors.newFixedThreadPool()` 이 이것을 무계로 쓰기 때문에, 소비 속도가 생산 속도를 따라가지 못하면 큐가 계속 자라다가 OOM 으로 죽는다
- 스레드 풀을 직접 생성하도록 권하는 이유가 여기에 있다

구조적 특징은 `two-lock queue` 알고리즘이다.

`putLock` 과 `takeLock` 이 분리되어 있어서 생산자와 소비자가 서로 경합하지 않는다.
대신 원소마다 Node 객체를 새로 할당하므로 GC 압력이 생긴다.

ArrayBlockingQueue : 배열 기반, 유계 FIFO

- 고정 크기 원형 배열이다
- 생성 시점에 메모리가 확정되고, 원소당 추가 할당이 발생하지 않는다
- 대신 락이 하나라서 생산자와 소비자가 같은 락을 두고 경합한다

그래서 경합이 심하고 처리량이 중요하다면 `LinkedBlockingQueue` 를,
메모리 예측 가능성과 낮은 오버헤드가 중요하다면 `ArrayBlockingQueue` 를 쓴다.

PriorityBlockingQueue : 힙 기반, 무계 우선순위 큐

- FIFO 가 아니라 Comparable / Comparator 순서대로 나온다
- 무계이므로 `put()` 은 절대 블로킹되지 않는다
- 블로킹은 `take()` 쪽에서만 발생한다

우선순위가 같은 원소 간에는 순서 보장이 없다.

FIFO 를 원하면 시퀀스 번호를 `tie-breaker` 로 직접 넣어야 한다.

또한 우선순위가 낮은 원소의 기아(starvation)가 구조적으로 가능하다.

DelayQueue : 힙 기반, 시간 기반 스케줄링 큐

- 원소가 `Delayed` 를 구현해야 하고, `getDelay()` 가 0 이하가 된 원소만 `take()` 할 수 있다
- 내부적으로 만료 시각 기준 우선순위 큐이며, 만료 전이면 큐에 원소가 있어도 소비자는 대기한다

SynchronousQueue : 랑데부(rendezvous) 메커니즘

- 용량이 0이라 저장 공간 자체가 없으며, `size()` 는 항상 0, `isEmpty()` 는 항상 true 이다
- `put()` 은 다른 스레드가 `take()` 를 호출할 때까지 블로킹되고, 그 순간 직접 핸드오프가 일어난다

구현체	유계	순서	락 구조	대표 용처
LinkedBlockingQueue	선택적	FIFO	이중 락	FixedThreadPool
ArrayBlockingQueue	유계	FIFO	단일 락	배압이 필요한 생산자-소비자
PriorityBlockingQueue	무계	우선순위	단일 락	작업 우선순위 처리
DelayQueue	무계	만료 시각	단일 락	스케줄링, 캐시 만료
SynchronousQueue	0	핸드오프	락프리	CachedThreadPool

TransferQueue : Interface, JDK 7

- 특화된 **BlockingQueue** 로, 큐에 **원소를 추가하는 코드가 다른 스레드의 코드가 그 원소를 꺼낼 때까지 대기한다.**
- 단순히 큐에 넣고 끝나는 것이 아니라 **소비자가 실제로 그 원소를 가져갈 때까지 생산자가 기다릴 수 있다는 점**이 일반 블로킹 큐와의 차이이다.

이 대기 동작은 **transfer(e)** 메서드가 담당하며, 기존의 **put(e)** 은 그대로 대기 없이 반환된다.

TransferQueue 의 단일 구현체는 **LinkedTransferQueue** 이며, 연결 노드에 기반한 **무계 TransferQueue** 이다.

2.5 Deque Implementations

Deque 인터페이스는 양 끝점 모두에서 원소의 삽입, 제거, 조회를 지원한다

범용 구현체

범용 구현체로는 **ArrayDeque** 와 **LinkedList** 가 있다.

- **ArrayDeque** : 크기 조절이 가능한 배열로 Deque 인터페이스를 구현한다
- **LinkedList** : List 구현체이면서 Deque 인터페이스도 함께 구현한다

기본 연산은 양 끝에 대한 삽입, 삭제, 조회이다.

addFirst, **addLast**, **removeFirst**, **removeLast**, **getFirst**, **getLast** 가 이에 해당하며, **addFirst** 는 head 에, **addLast** 는 tail 에 원소를 추가한다.

유연성 측면에서는 **LinkedList** 가 **ArrayDeque** 보다 더 유연하다.

LinkedList 는 모든 선택적 리스트 연산을 구현하며 null 원소를 허용한다.

반면 **ArrayDeque** 는 null 원소를 허용하지 않는다.

효율성 측면에서는 **ArrayDeque** 가 양 끝의 추가-제거 연산에서 **LinkedList** 보다 더 효율적이다.

다만 순회 도중 현재 원소를 제거하는 작업은 **LinkedList** 쪽이 더 낫다.

LinkedList 자체는 순회에 이상적인 구조가 아니라는 점은 유의해야 한다.

메모리 측면에서는 LinkedList 구현이 ArrayDeque 구현보다 메모리를 더 소비한다.

ArrayDeque 인스턴스의 순회에는 두 가지 방식이 제시된다.

: forEach, Iterator

```
java

ArrayDeque<String> aDeque = new ArrayDeque<String>();

// foreach
for (String str : aDeque) {
    System.out.println(str);
}

// Iterator
for (Iterator<String> iter = aDeque.iterator(); iter.hasNext(); ) {
    System.out.println(iter.next());
}
```

LinkedList 와 ArrayDeque 모두 다중 스레드에 의한 접근을 지원하지 않으며, 이 제약을 해결하려면 동시성 구현체가 필요하다

동시성 구현체

LinkedBlockingDeque : Deque 인터페이스의 동시성 구현체

Deque 가 비어 있으면 **takeFirst** , **takeLast** 같은 메서드가 원소가 사용 가능해질 때까지 대기한 뒤, 그 원소를 조회하고 제거한다.